应用可观测性技术建设

美团 基础软件研发中心/监控系统组 颜成吉

美团

目录

1. 可观测性体系介绍

2. 可观测性体系建设

3. 可观测性方向探索

4. 总结与展望

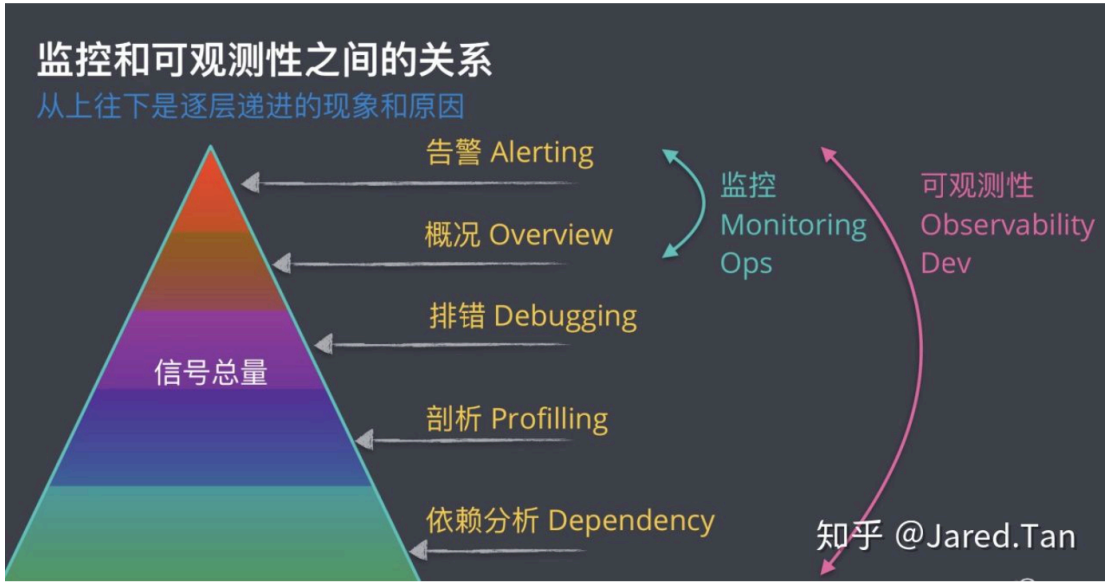
可观测性与监控的关系?

- Baron SchSchwarz 大咖的一个定义:
  监控告诉我们系统的哪些部分是工作的，可观测性告诉我们那里为什么不工作了。

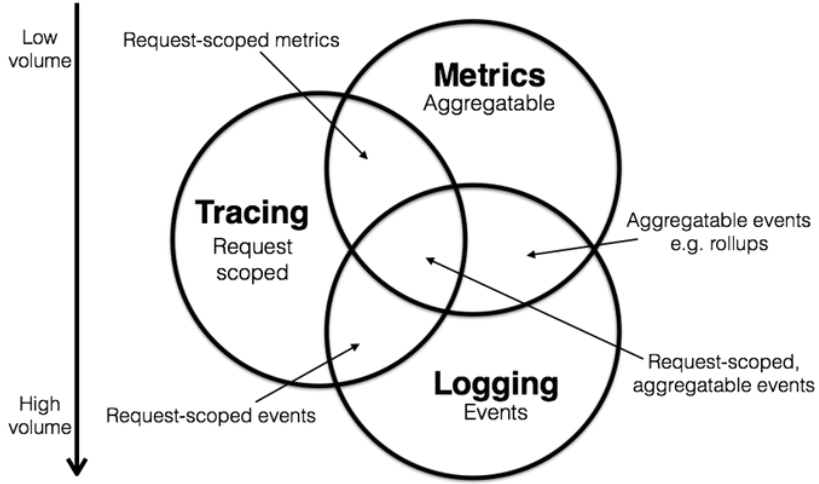
- 可观测性是监控的超集

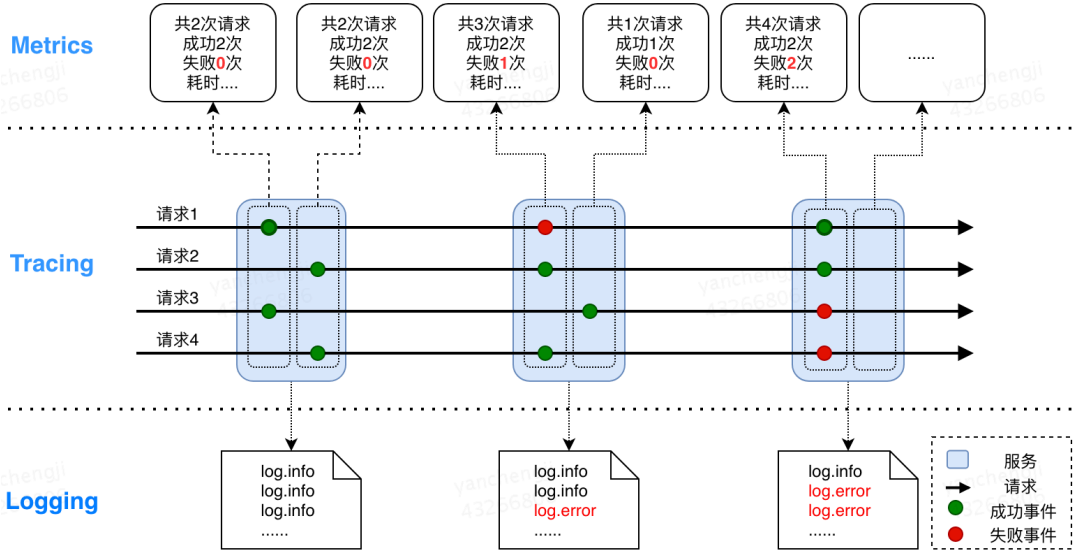
- 监控侧重宏观，可观测包括微观能力

- 可观测性三大支柱:
  - **Metrics、Logging、Tracing**

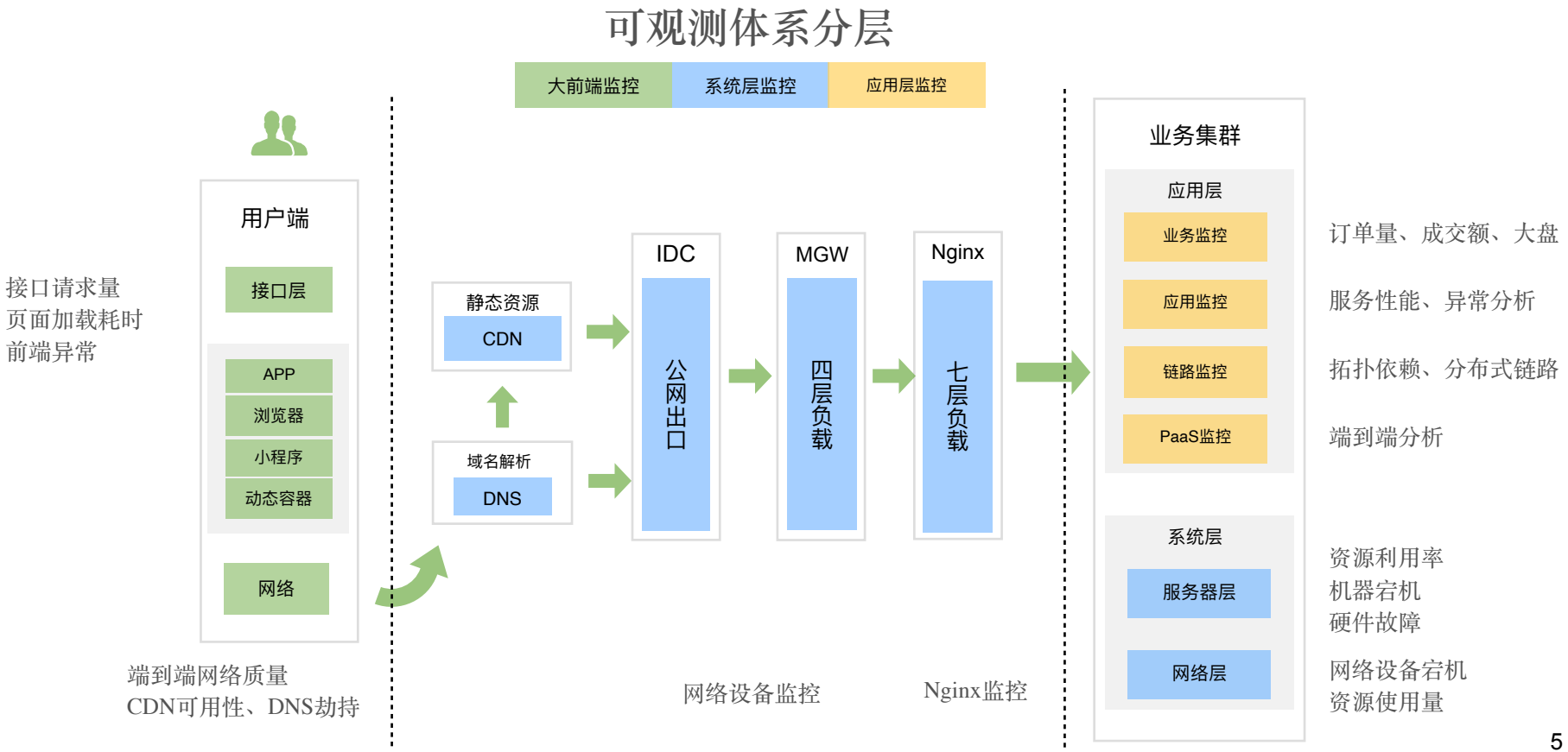


可观测体系—三大支柱


Low volume
High volume
Request-scoped metrics
Metrics
Aggregatable
Tracing
Request scoped
Logging
Events
Aggregatable events e.g. rollups
Request-scoped, aggregatable events
Request-scoped events


Metrics
共2次请求 成功2次 失败0次 耗时....
共2次请求 成功2次 失败0次 耗时....
共3次请求 成功2次 失败1次 耗时....
共1次请求 成功1次 失败0次 耗时....
共4次请求 成功2次 失败2次 耗时....
.....
Tracing
请求1
请求2
请求3
请求4
Logging
log.info log.info log.info .....
log.info log.info log.error .....
log.info log.error log.error .....
服务
请求
成功事件
失败事件

可观测体系—领域分层


可观测体系分层
大前端监控
系统层监控
应用层监控
用户端
接口层
APP
浏览器
小程序
动态容器
网络
接口请求量
页面加载耗时
前端异常
端到端网络质量
CDN可用性、DNS劫持
静态资源
CDN
域名解析
DNS
IDC
公网出口
MGW
四层负载
Nginx
七层负载
网络设备监控
Nginx监控
业务集群
应用层
业务监控
应用监控
链路监控
PaaS监控
系统层
服务器层
网络层
订单量、成交额、大盘
服务性能、异常分析
拓扑依赖、分布式链路
端到端分析
资源利用率
机器宕机
硬件故障
网络设备宕机
资源使用量

目录

1. 可观测性体系介绍

2. 可观测性体系建设

3. 可观测性方向探索

4. 总结与展望

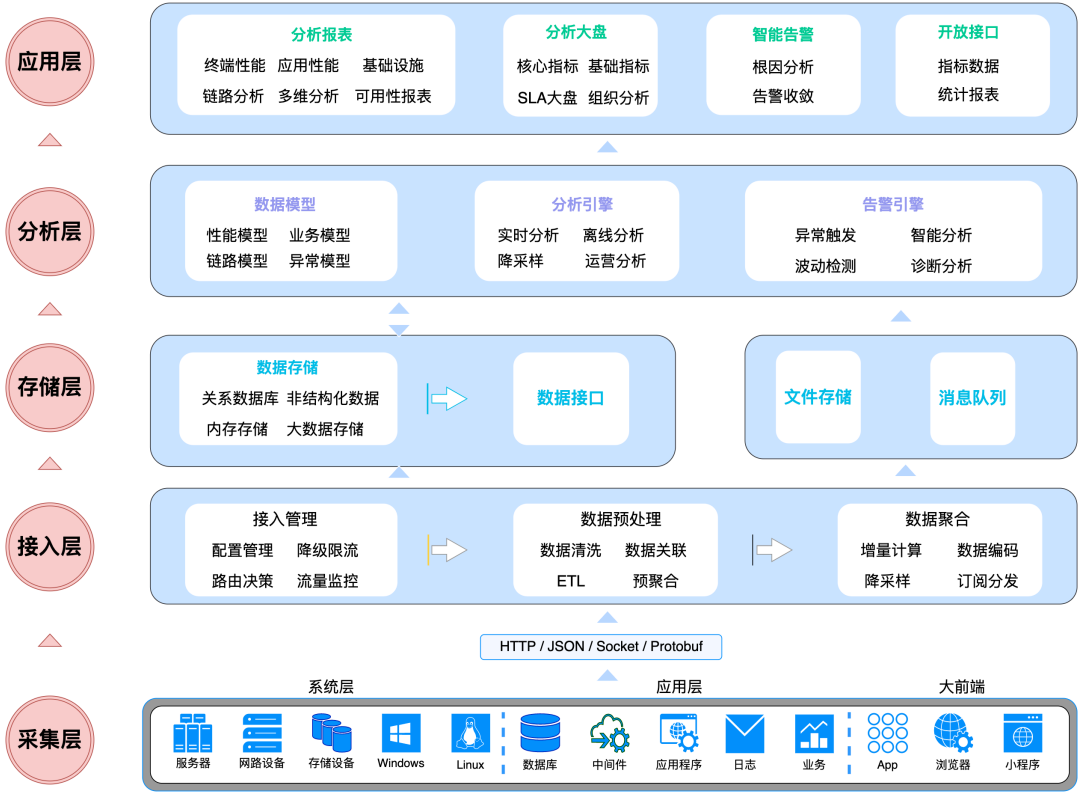
可观测性体系建设—Raptor监控平台

用户：全栈（SRE、RD、PM、QA等）　　领域：大前端、应用层、系统层

可观测性体系建设—Raptor顶层设计

设计原则：

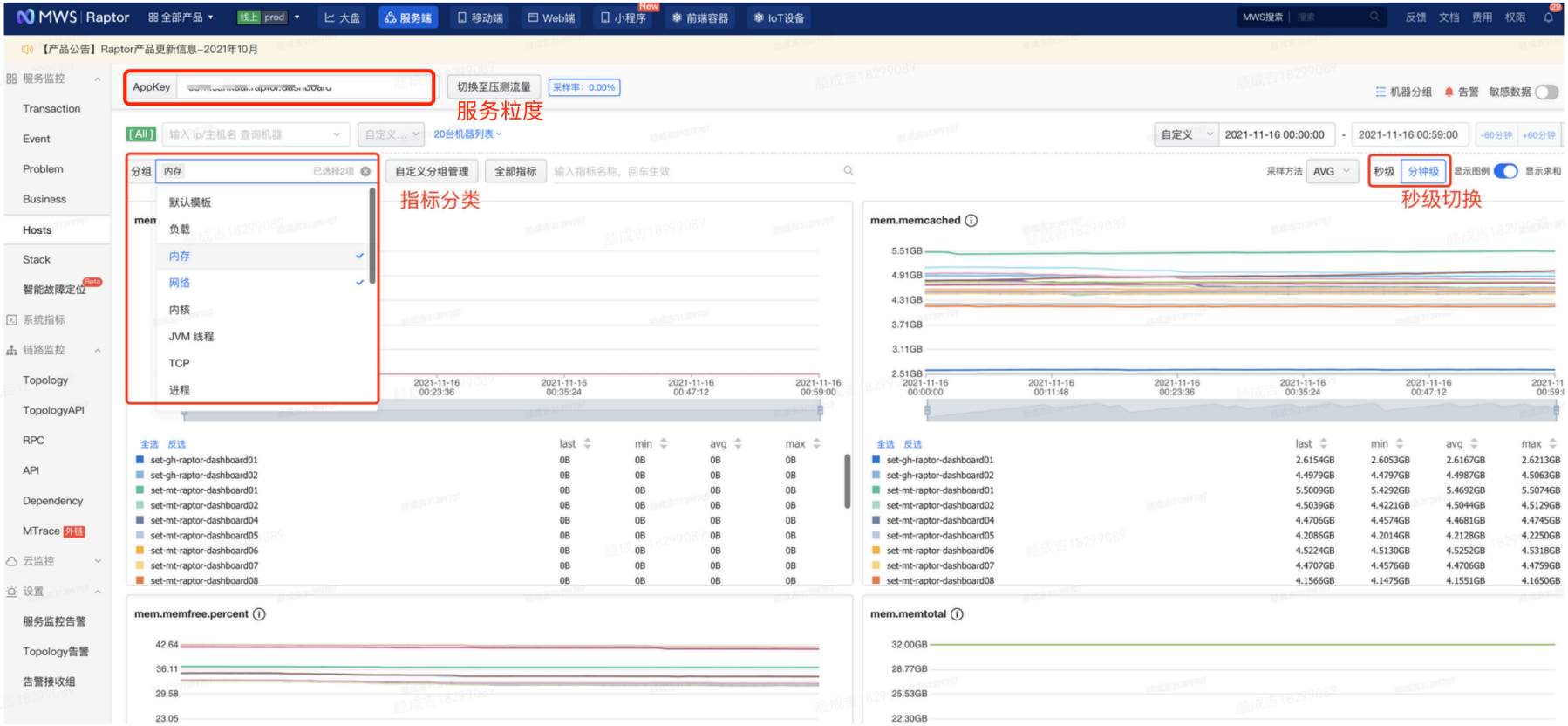
- ✓ 一体化解决方案，领域建模
- ✓ 分层设计
  - 低耦合，引擎可替换
  - 采集、接入、处理，严格分层
- ✓ 高并发
  - 横向：可扩展、去热点、分布式
  - 纵向：异步化、无锁化
- ✓ 高可用
  - 服务无状态化
  - 服务保护、故障演练
- ✓ 高性能
  - 批量、压缩、编码、预加载



可观测性体系建设—Raptor产品介绍

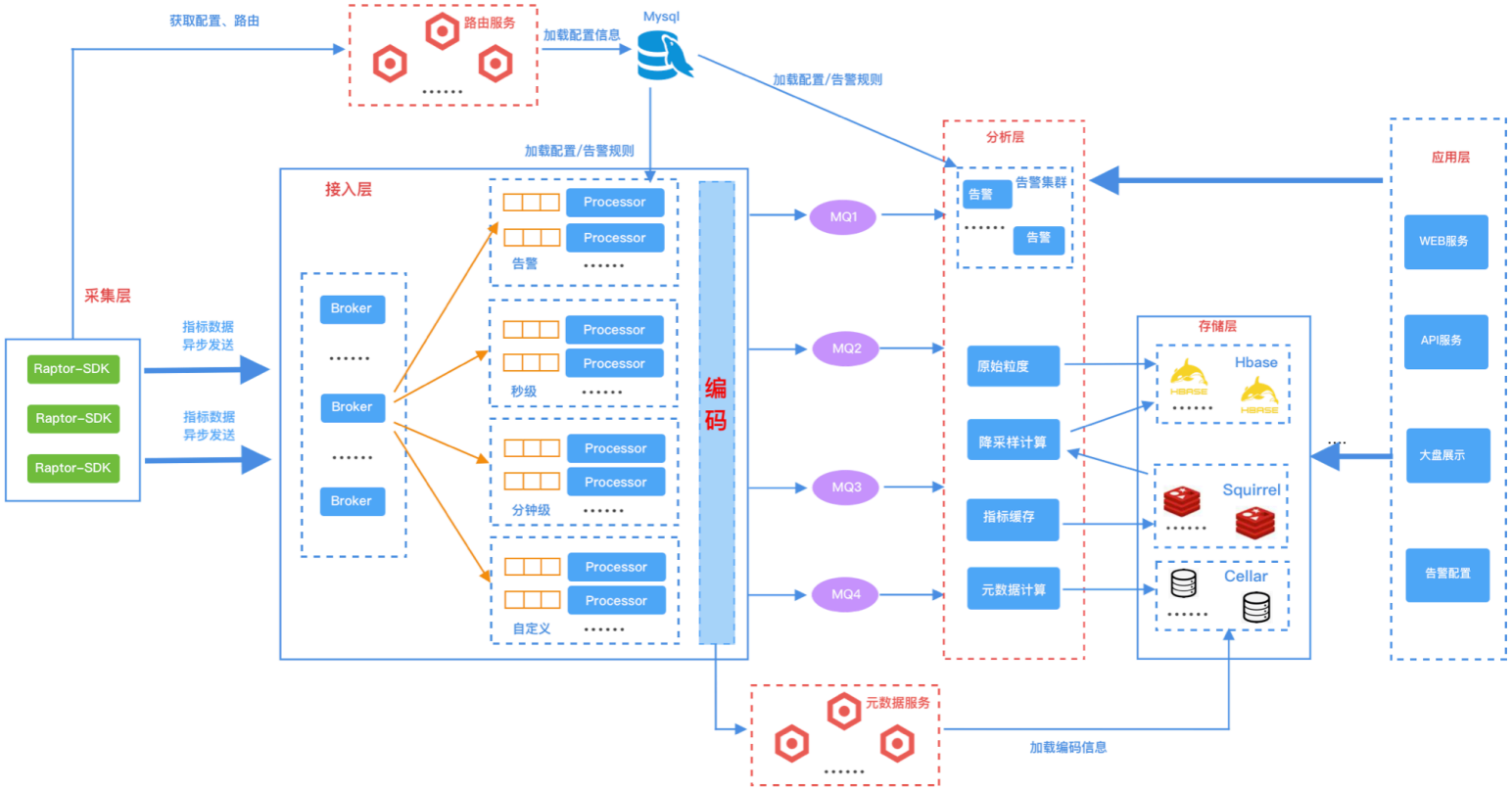
1. 系统监控
2. 链路监控
3. 应用监控
4. 大盘和告警

系统监控


MWS Raptor
全部产品
线上 prod
大盘
服务端
移动端
Web端
小程序
前端容器
IoT设备
【产品公告】Raptor产品更新信息--2021年10月
服务监控
Transaction
Event
Problem
Business
Hosts
Stack
智能故障定位
系统指标
链路监控
Topology
TopologyAPI
RPC
API
Dependency
MTrace
云监控
设置
服务监控告警
Topology告警
告警接收组
AppKey
切换至压测流量
采样率：0.00%
服务粒度
机器分组
告警
敏感数据
[All]
输入 ip/主机名 查询机器
自定义...
20台机器列表
自定义
2021-11-16 00:00:00
2021-11-16 00:59:00
分组
内存
已选择2项
默认模板
负载
内存
网络
内核
JVM 线程
TCP
进程
自定义分组管理
全部指标
输入指标名称，回车生效
指标分类
采样方法
AVG
秒级
分钟级
显示图例
显示求和
秒级切换
mem.memcached
5.51GB
4.91GB
4.31GB
3.71GB
3.11GB
2.51GB
全选 反选
last
min
avg
max
set-gh-raptor-dashboard01
set-gh-raptor-dashboard02
set-mt-raptor-dashboard01
set-mt-raptor-dashboard02
set-mt-raptor-dashboard04
set-mt-raptor-dashboard05
set-mt-raptor-dashboard06
set-mt-raptor-dashboard07
set-mt-raptor-dashboard08
mem.memfree.percent
42.64
36.11
29.58
23.05
mem.memtotal
32.00GB
28.77GB
25.53GB
22.30GB

系统监控—架构设计

- 采集层
- 接入层
- 分析层
- 存储层
- 应用层



系统监控—存储设计

- 倒排索引

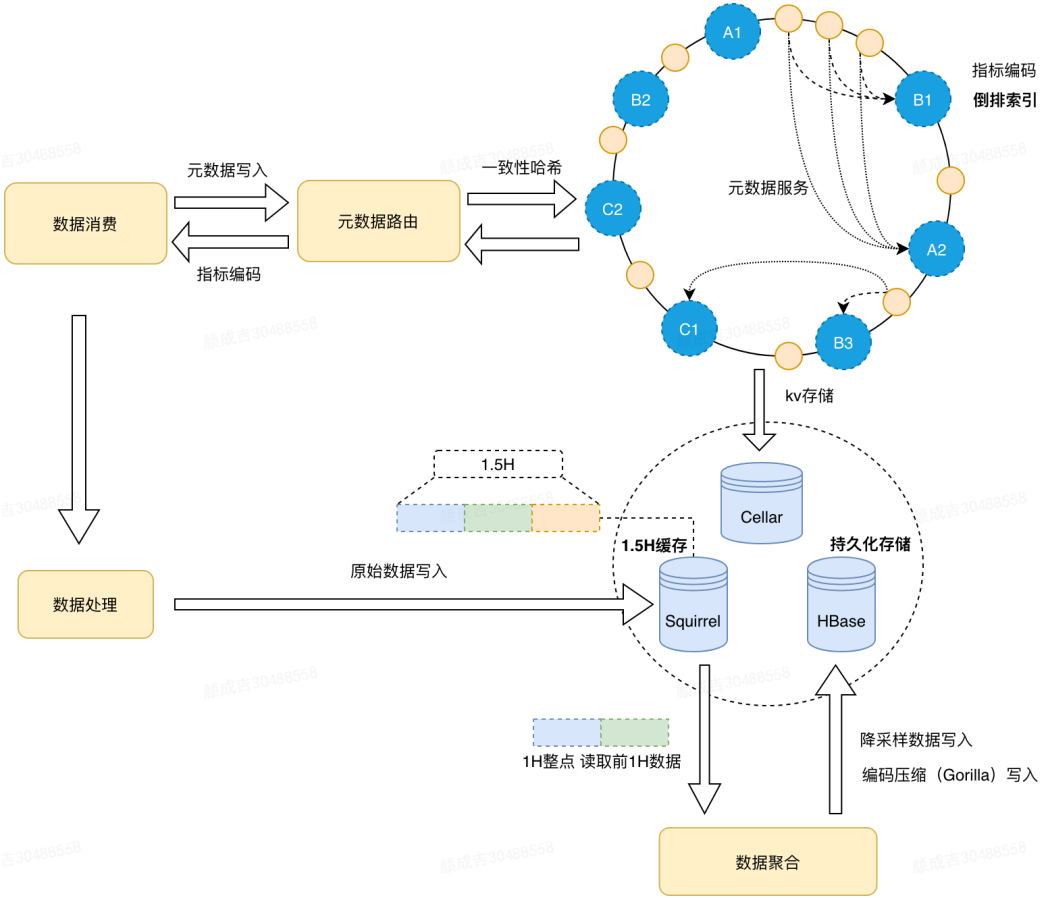
  一致性哈希，内存多副本，KV持久化

- 近实时缓存

  编码，多副本，1.5H窗口

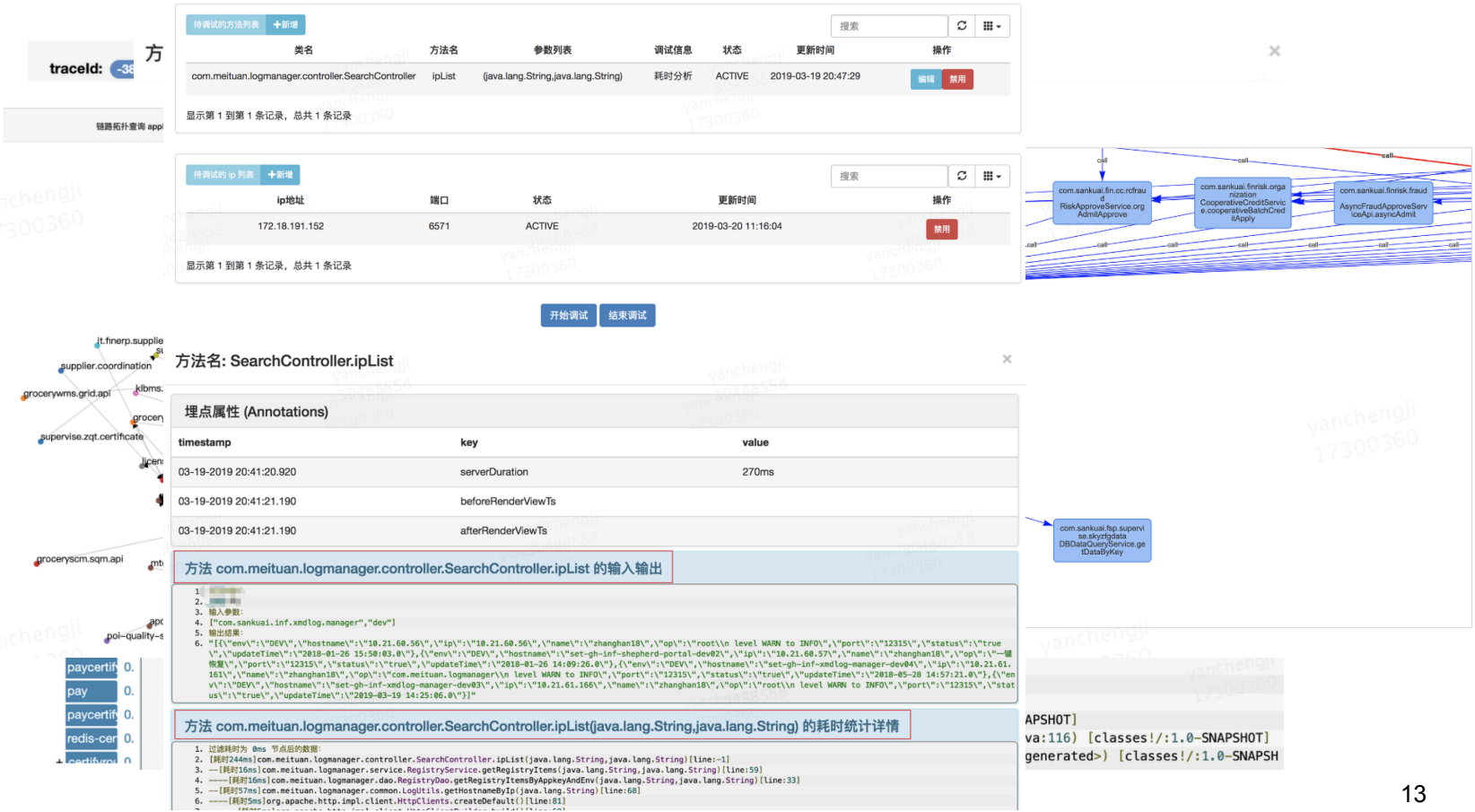
- 持久化存储

  编码，Rowkey打散，降采样



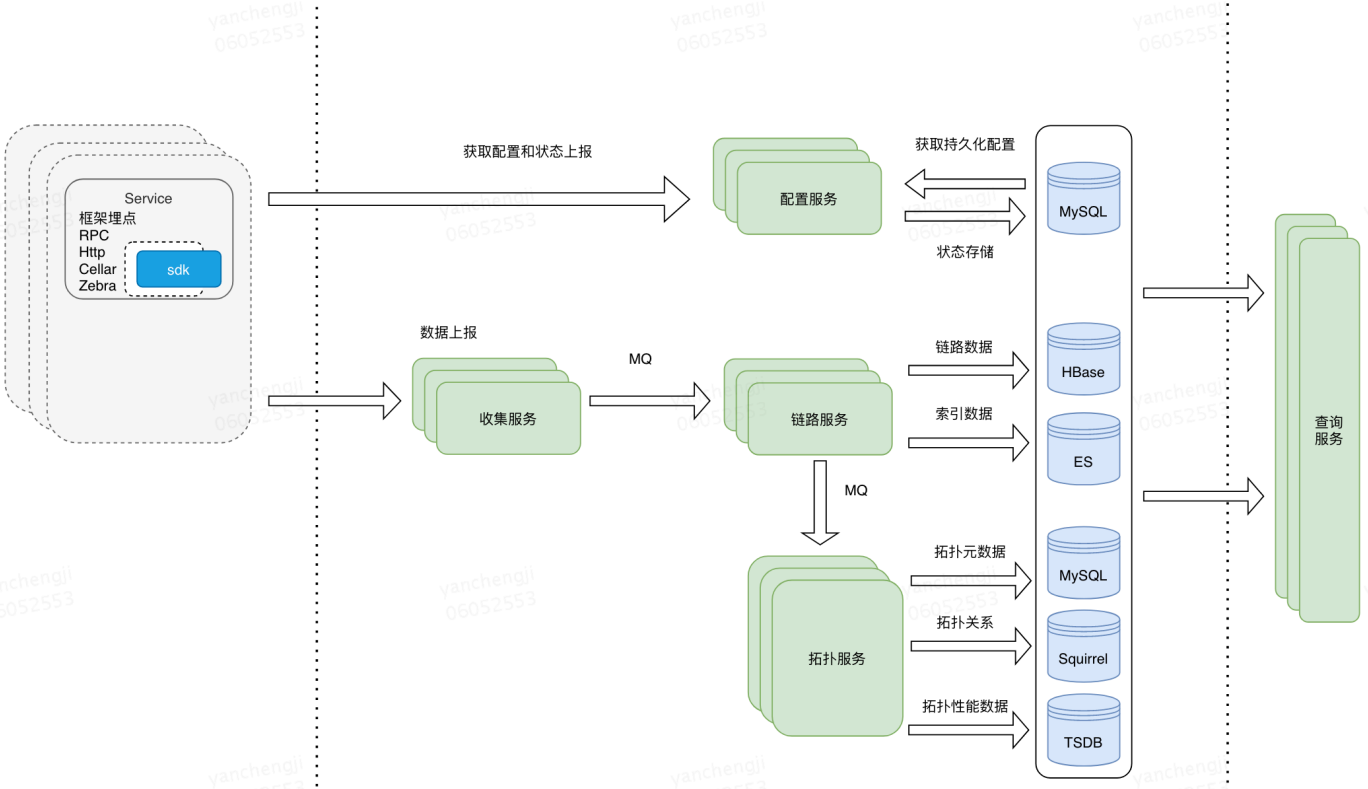
链路监控

- 请求链路
- 链路拓扑
- 远程调试



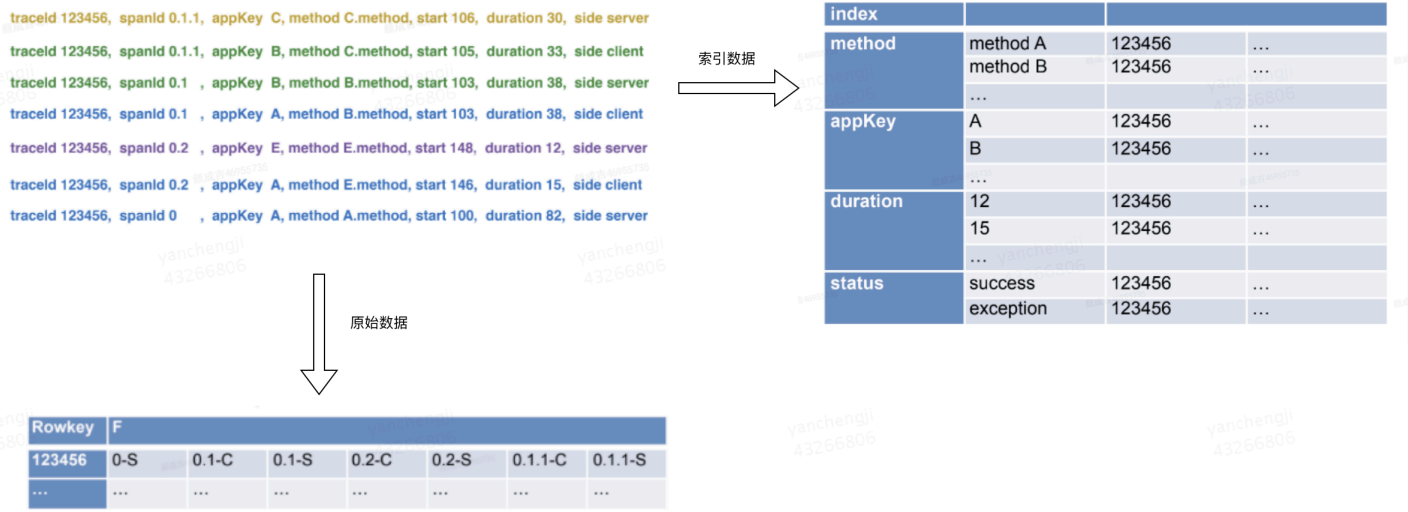
链路监控—架构设计

- 显式埋点
- 动态配置
- 链路处理
- 拓扑处理



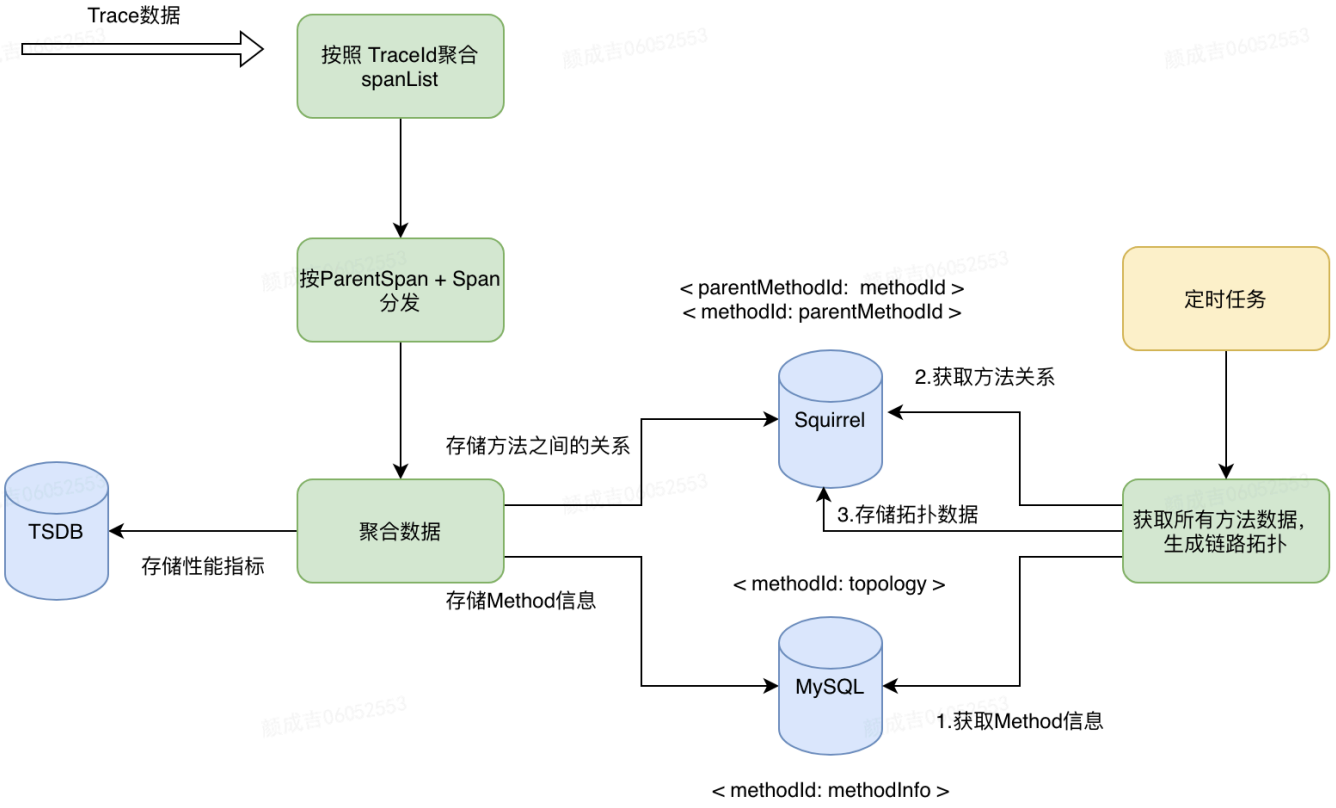
链路监控—链路存储

- HBase存储原始数据
- ES存储索引数据



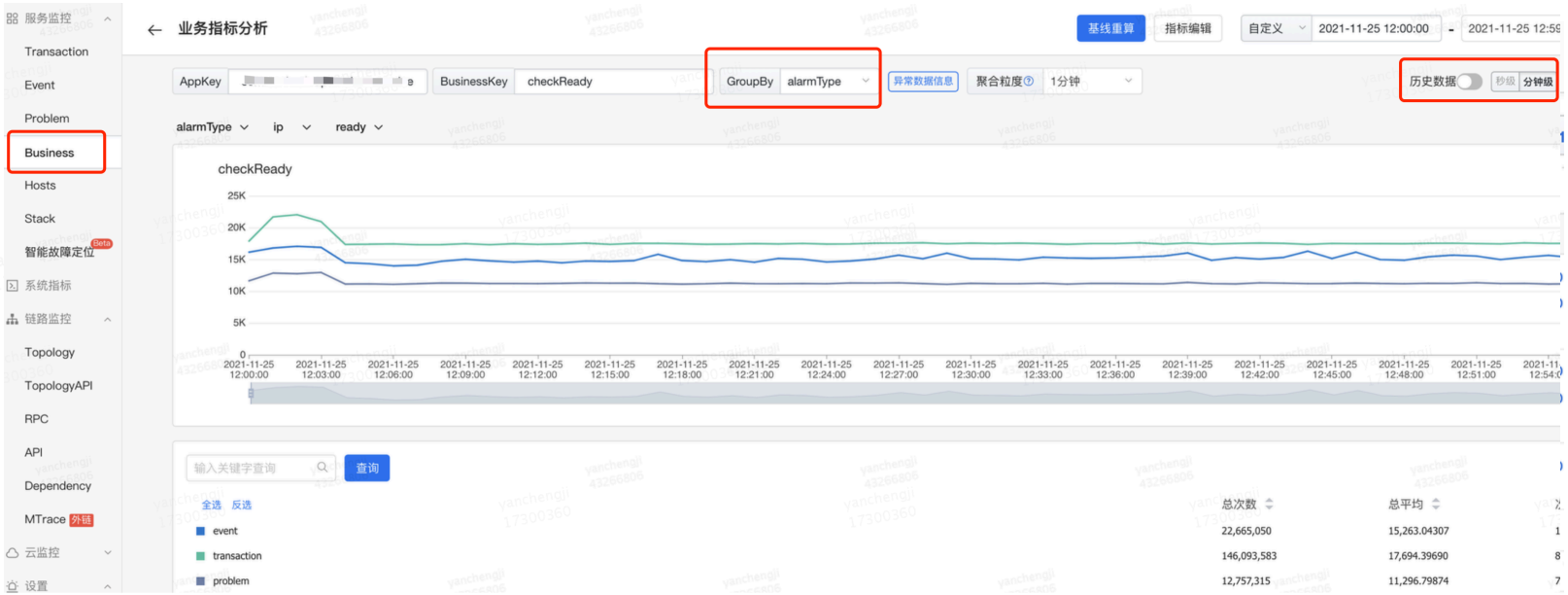
链路监控—拓扑计算

- 应用拓扑
- 方法拓扑



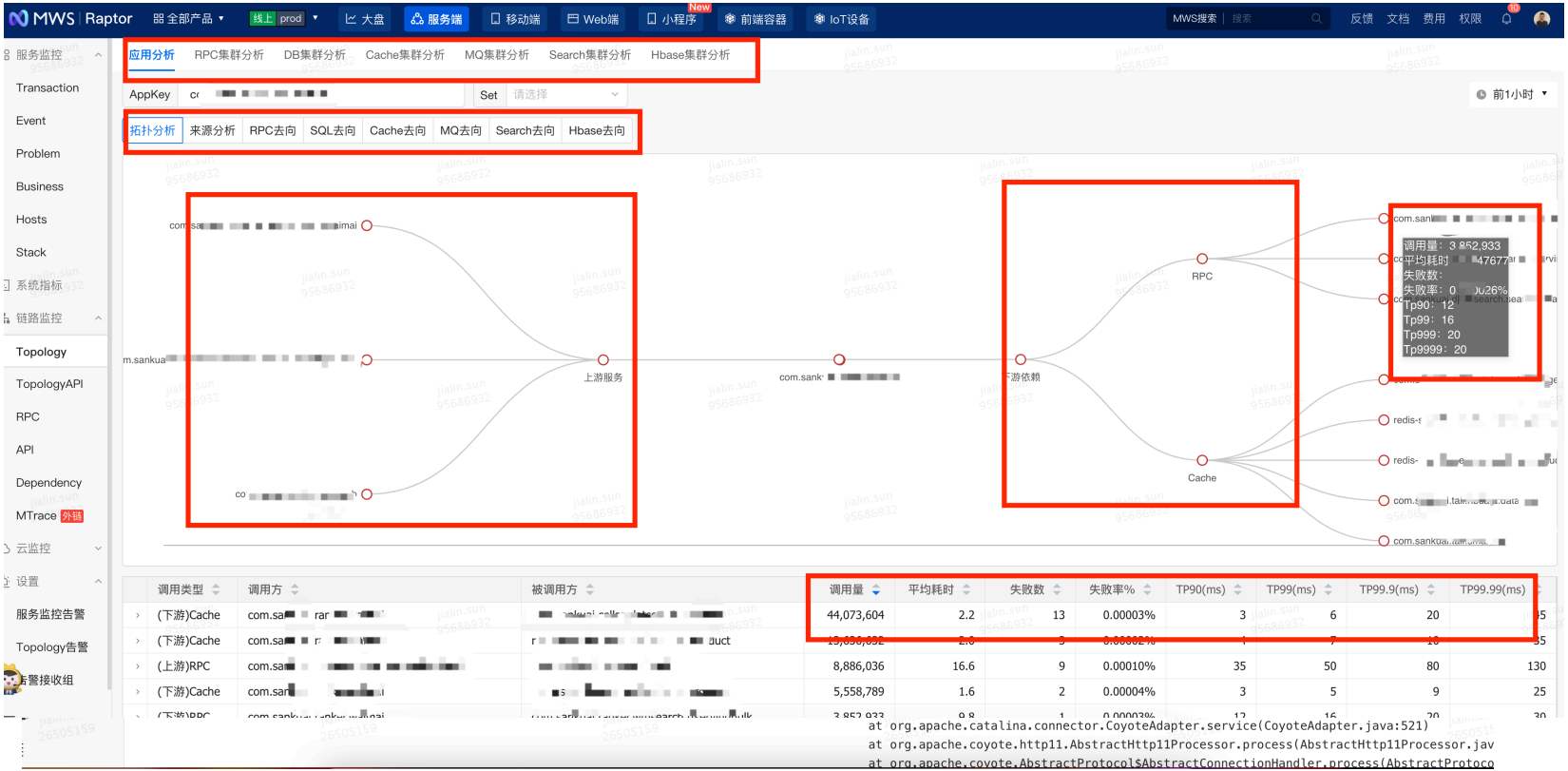
应用监控—指标部分

- 服务指标
- 异常分析
- 堆栈分析
- 业务指标



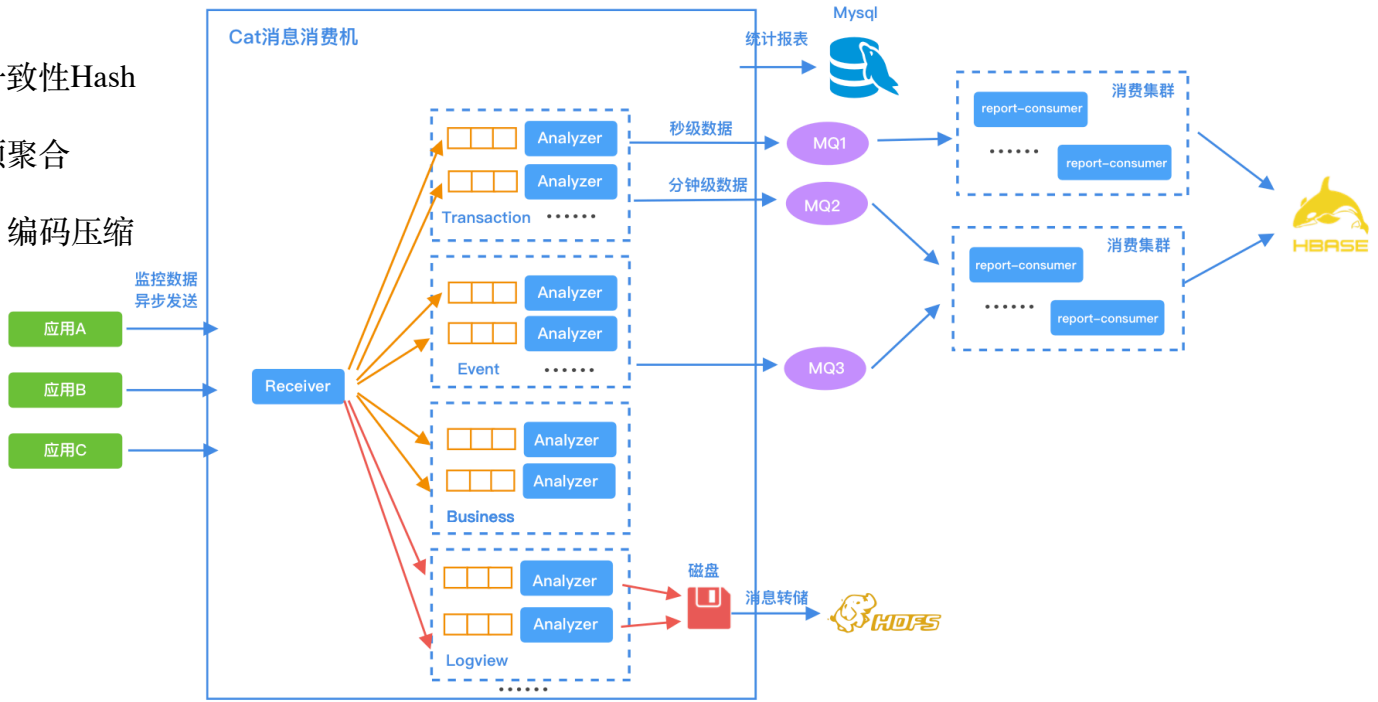
应用监控—链路部分

- 事务链路
- 集群链路

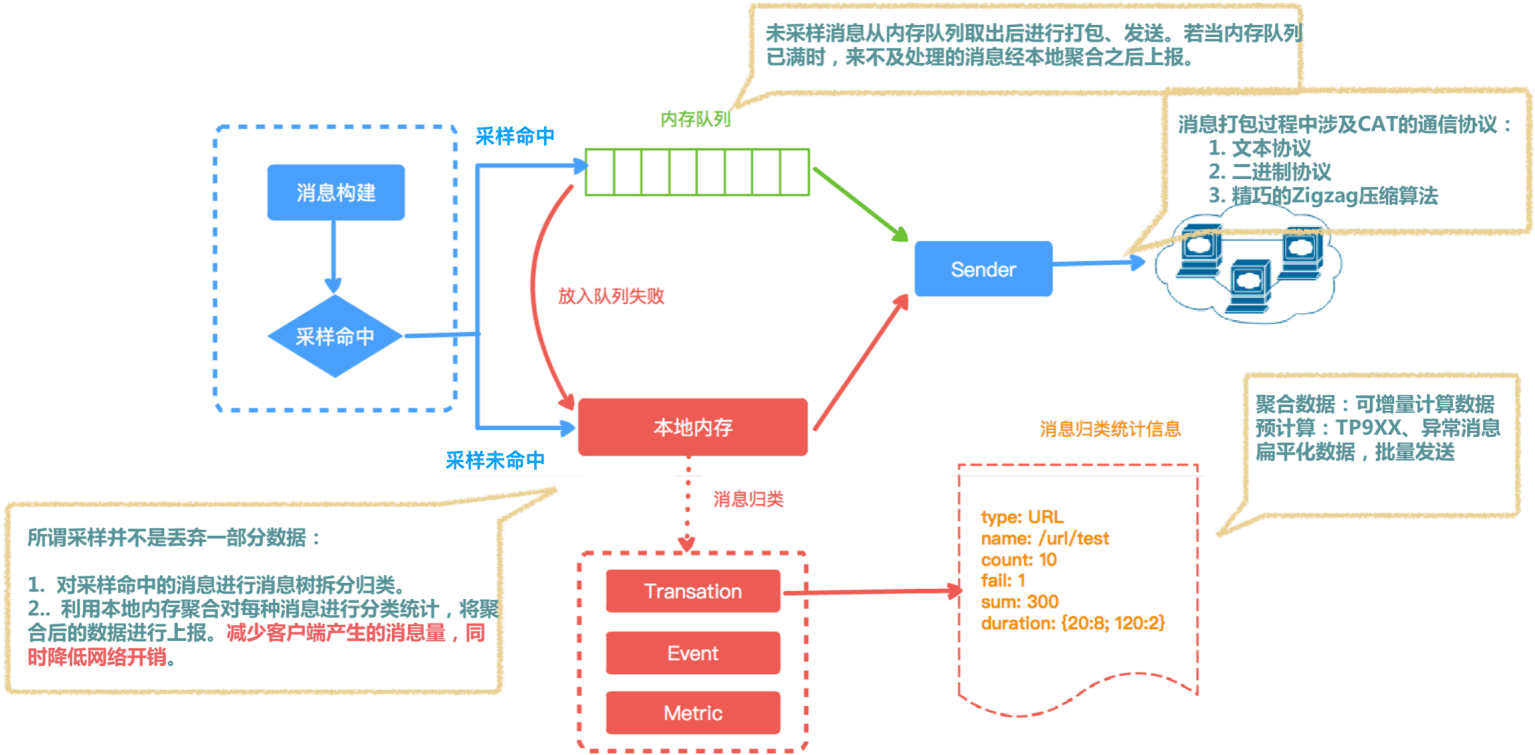


应用监控—架构设计

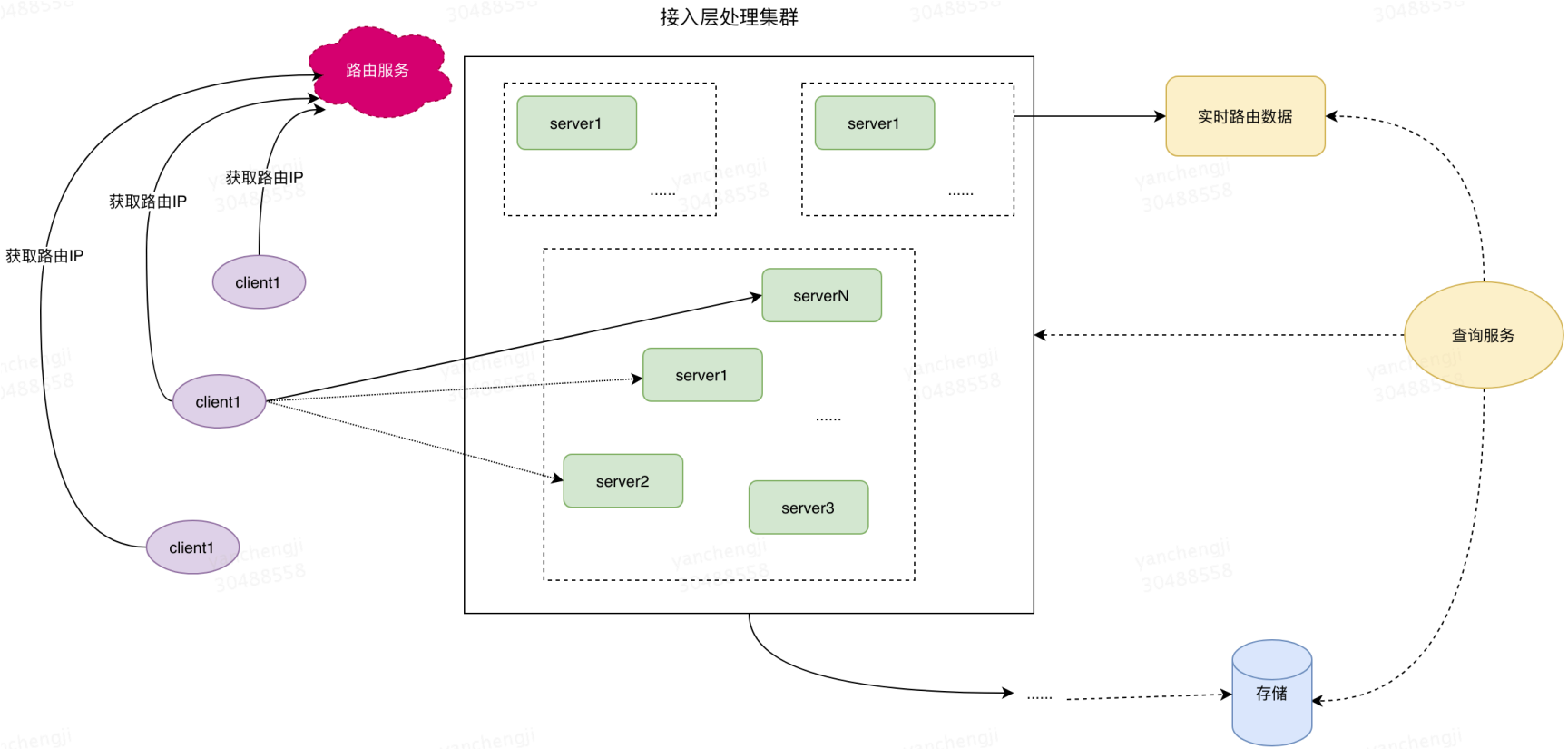
- 采集层：采样聚合
- 接入层：多集群，路由一致性Hash
- 分析层：清洗，规整，预聚合
- 存储层：缓存，降采样，编码压缩



应用监控—客户端设计


未采样消息从内存队列取出后进行打包、发送。若当内存队列已满时，来不及处理的消息经本地聚合之后上报。
内存队列
采样命中
消息构建
采样命中
放入队列失败
Sender
消息打包过程中涉及CAT的通信协议：
1. 文本协议
2. 二进制协议
3. 精巧的Zigzag压缩算法
本地内存
采样未命中
消息归类统计信息
聚合数据：可增量计算数据
预计算：TP9XX、异常消息
扁平化数据，批量发送
消息归类
所谓采样并不是丢弃一部分数据：
1. 对采样命中的消息进行消息树拆分归类。
2.. 利用本地内存聚合对每种消息进行分类统计，将聚合后的数据进行上报。减少客户端产生的消息量，同时降低网络开销。
Transation
Event
Metric
type: URL
name: /url/test
count: 10
fail: 1
sum: 300
duration: {20:8; 120:2}

应用监控—路由设计


接入层处理集群
路由服务
获取路由IP
获取路由IP
获取路由IP
client1
client1
client1
server1
.....
server1
.....
serverN
server1
.....
server2
server3
实时路由数据
查询服务
.....
存储

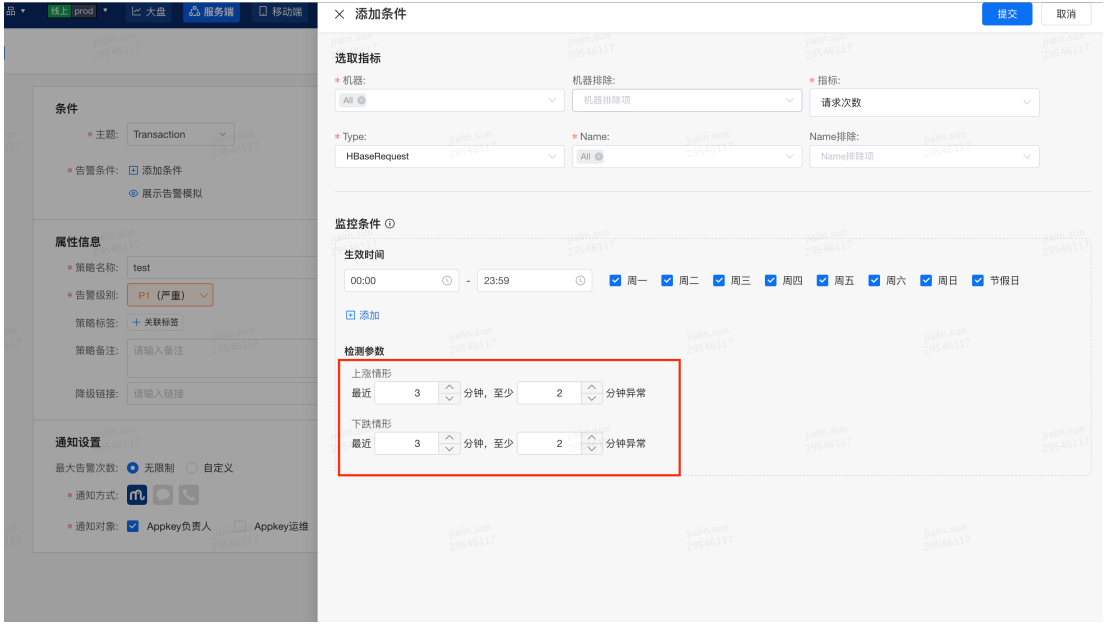
监控大盘

- 运营入口
- 多领域聚合
- 图表丰富
- 关联告警

统一告警

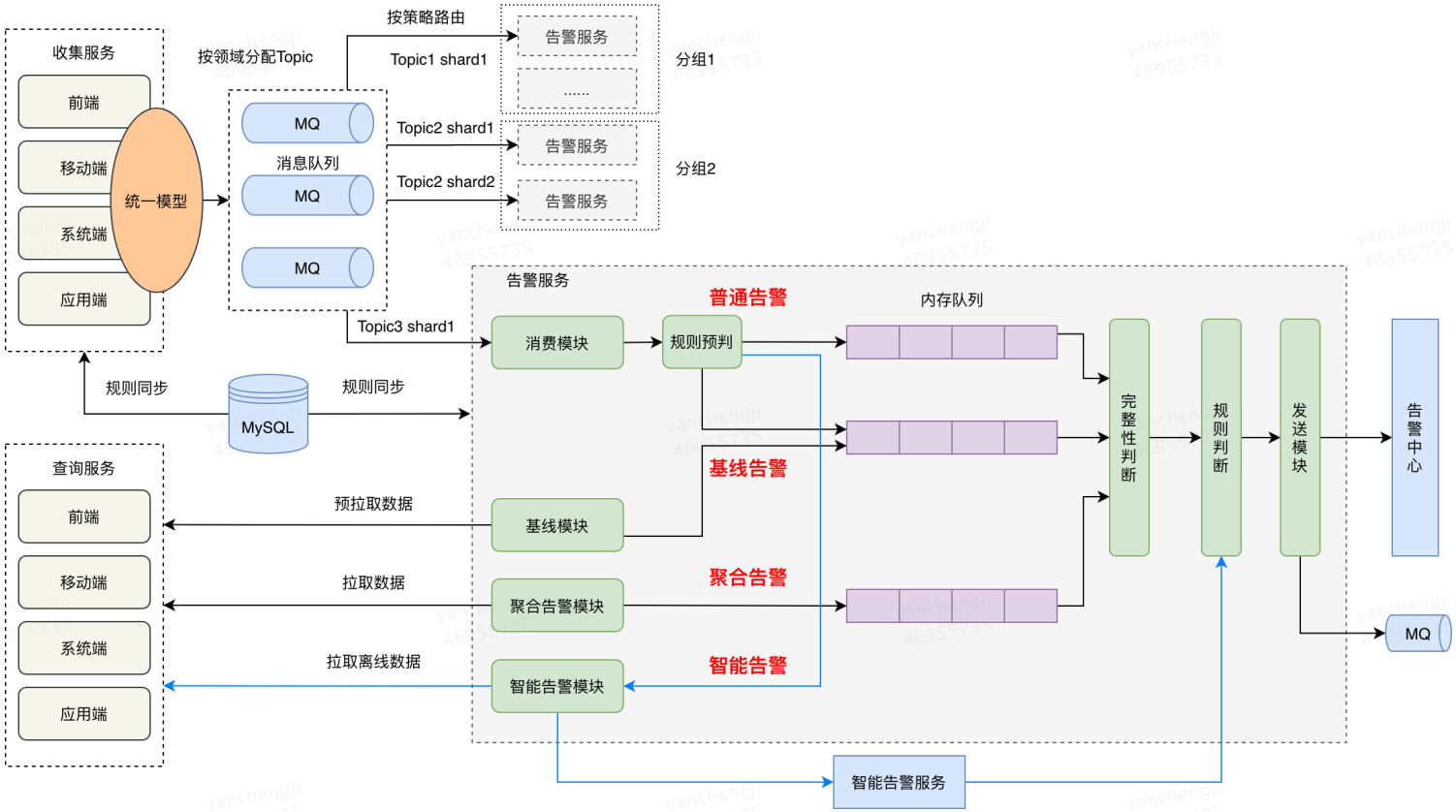
阈值告警

智能告警



统一告警—架构设计

- 统一模型
- 推拉结合
  - ✓ 普通告警
  - ✓ 基线告警
  - ✓ 聚合告警
  - ✓ 智能告警
- 告警订阅



1. 可观测性体系介绍

2. 可观测性体系建设

3. 可观测性方向探索

4. 总结与展望

可观测性方向探索

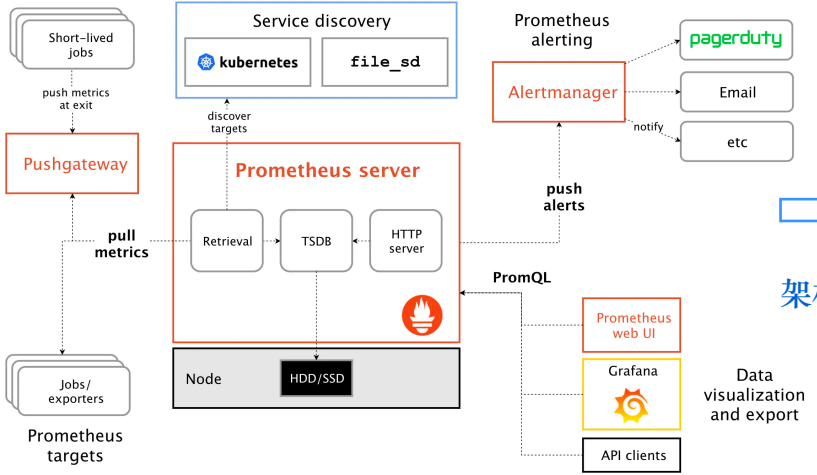
美团

拥抱云原生

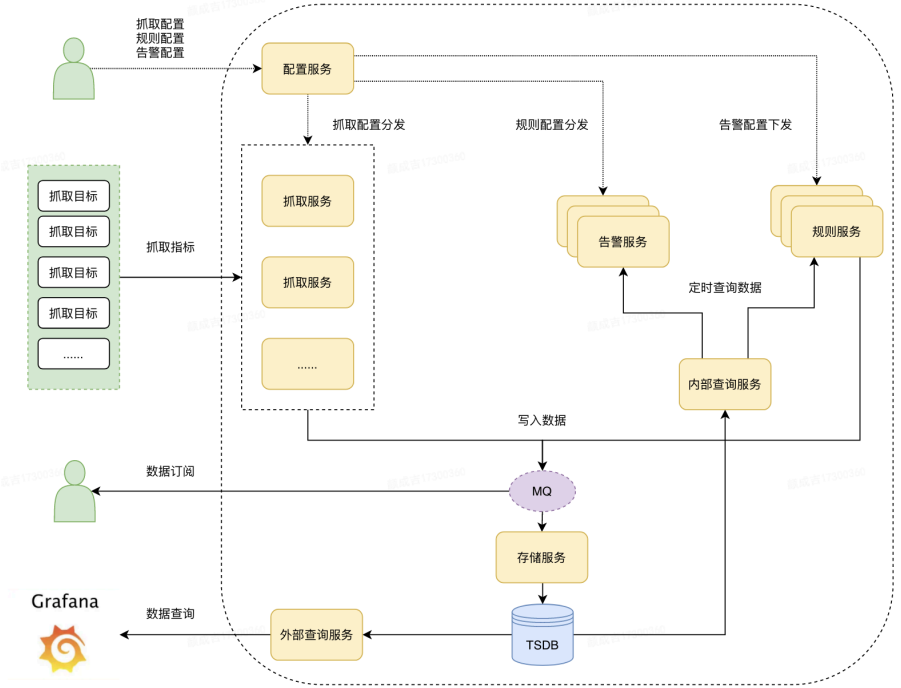
打破信息孤岛

挖掘信息价值

拥抱云原生—PrometheusService

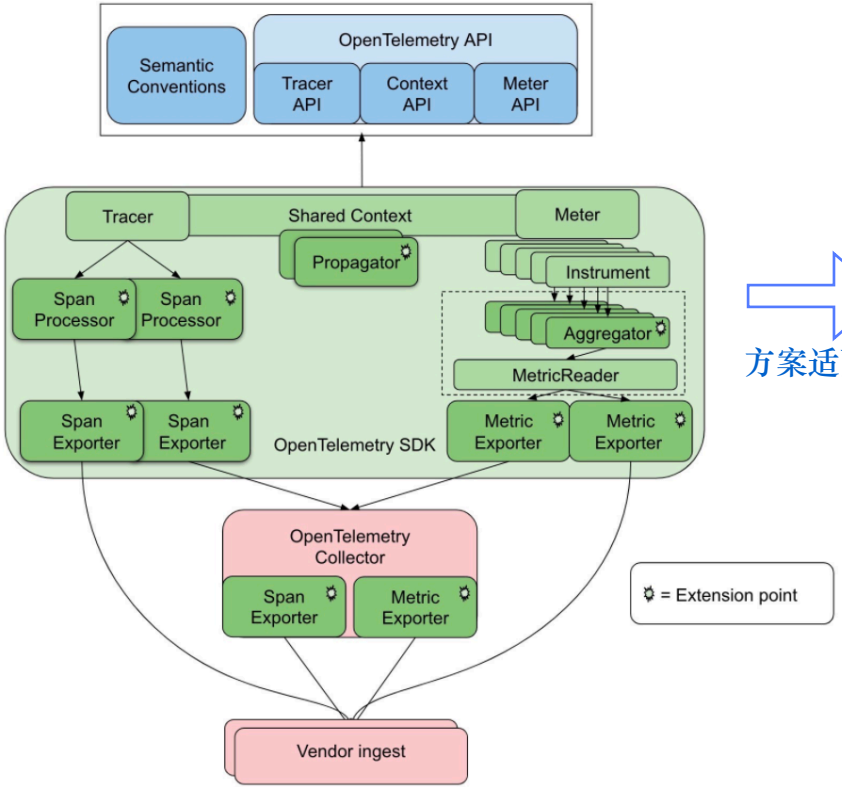


架构升级

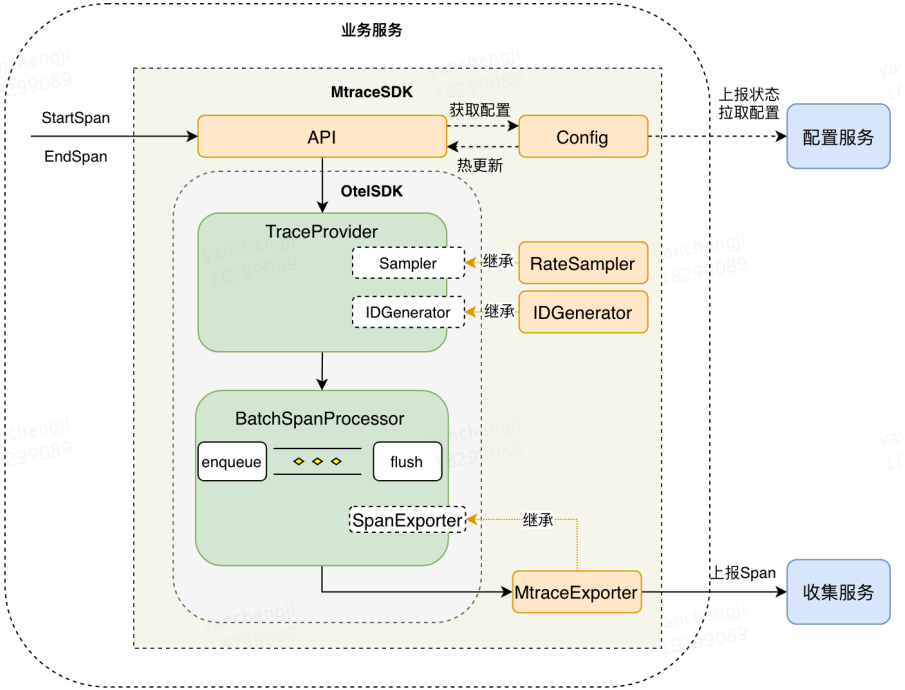


拥抱云原生—PrometheusService

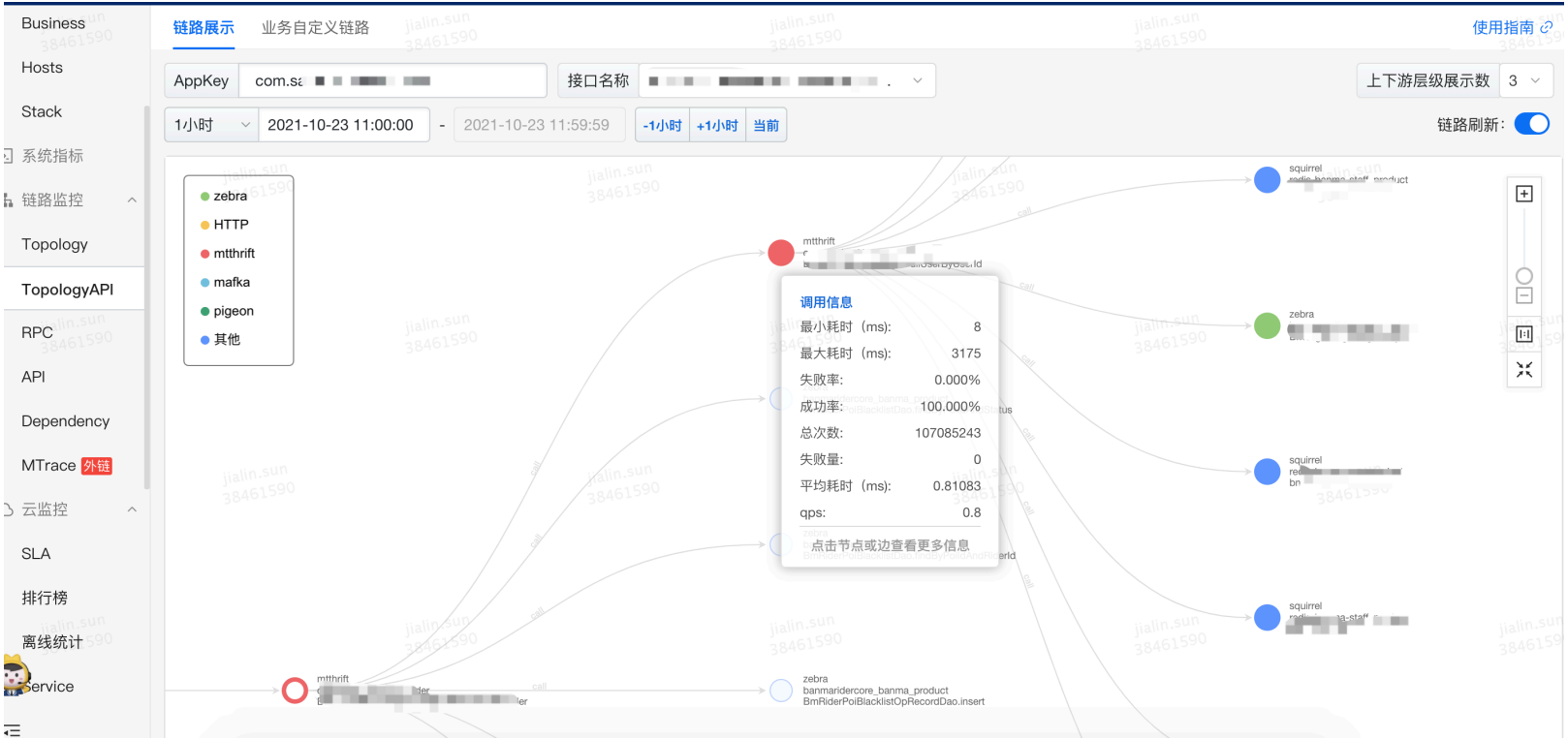
拥抱云原生—Tracing多语言SDK



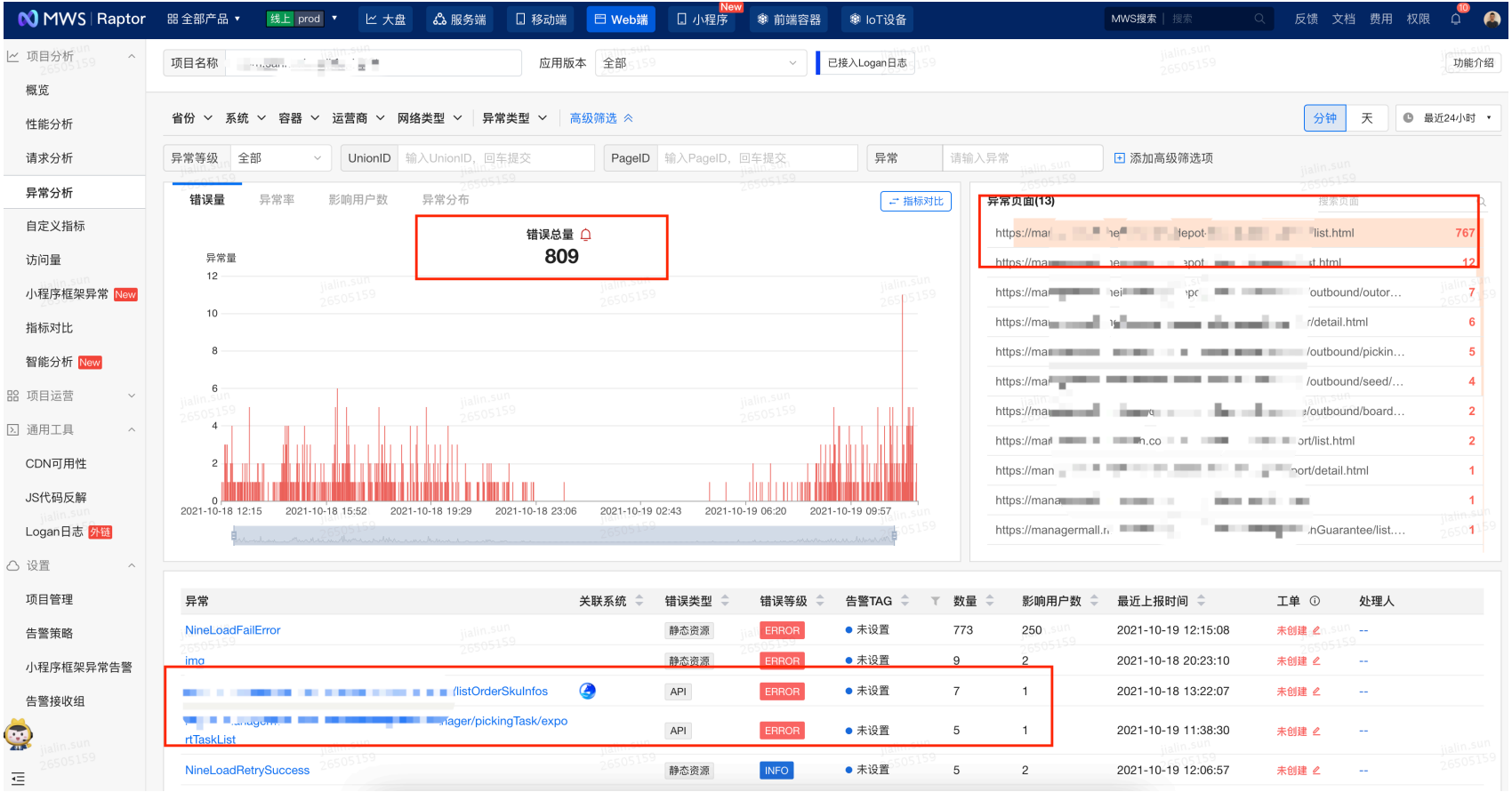
方案适配



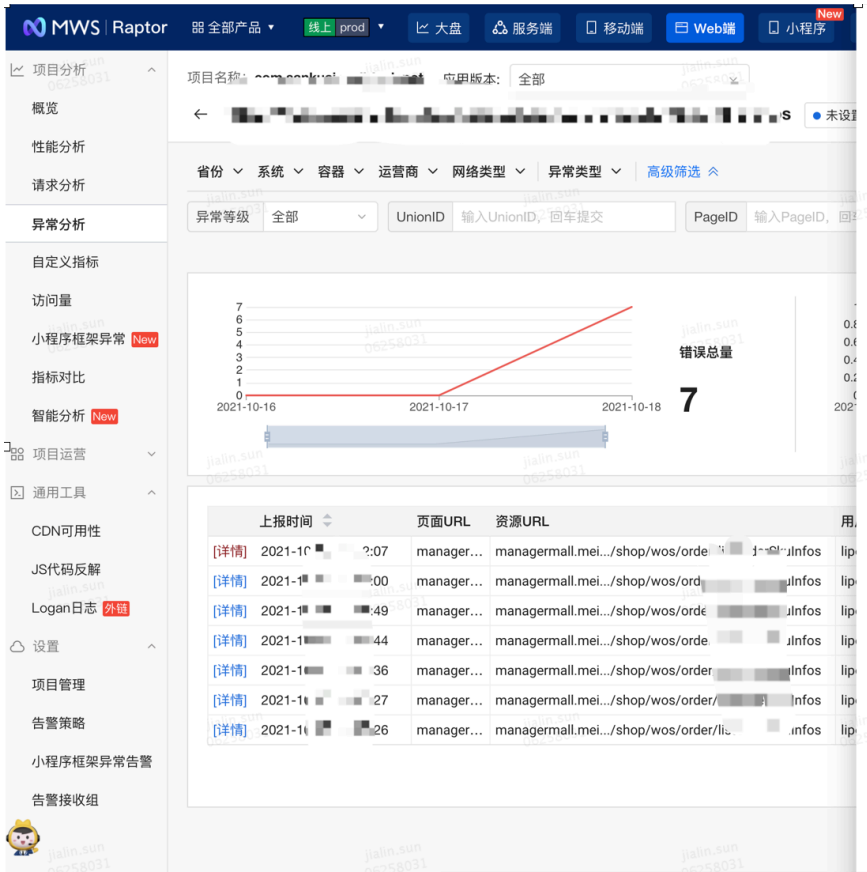
打破信息孤岛—后端Metrics和Tracing打通



打破信息孤岛—前后端链路打通

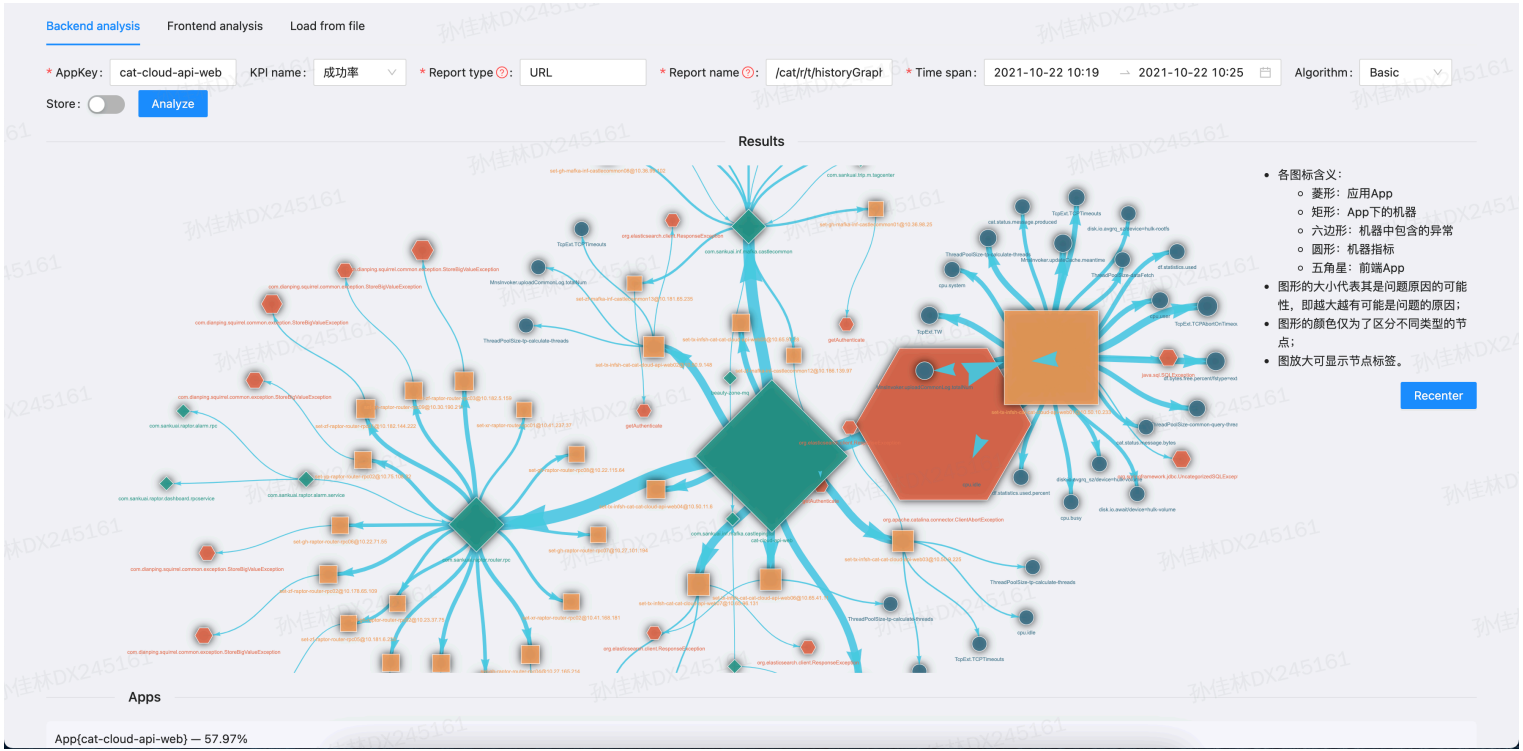


打破信息孤岛—前后端链路打通

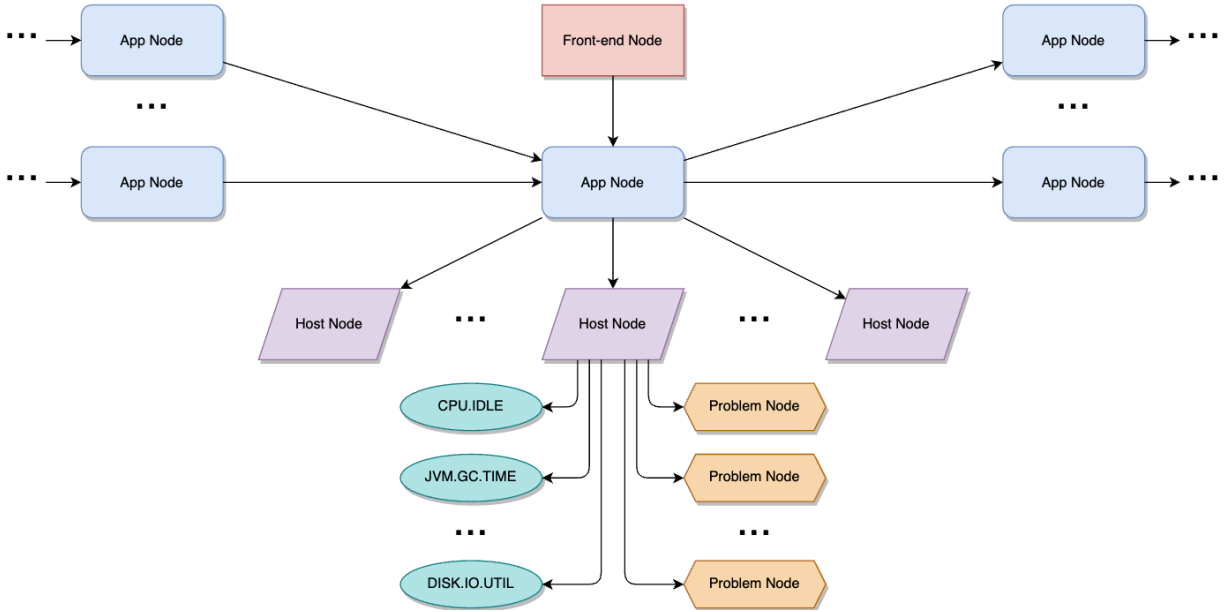


挖掘数据价值—异常根因分析

- 指标关联
- 根因分析



挖掘数据价值—异常根因分析



目录

1. 可观测性体系介绍

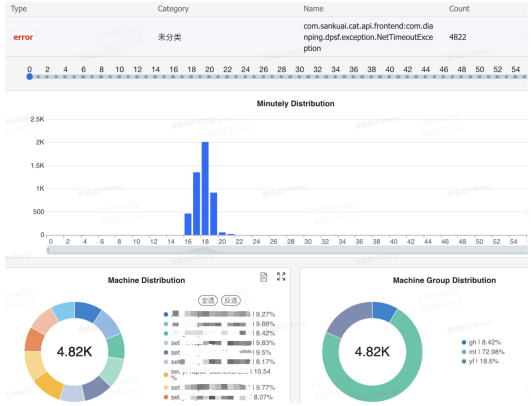
2. 可观测性体系建设

3. 可观测性方向探索

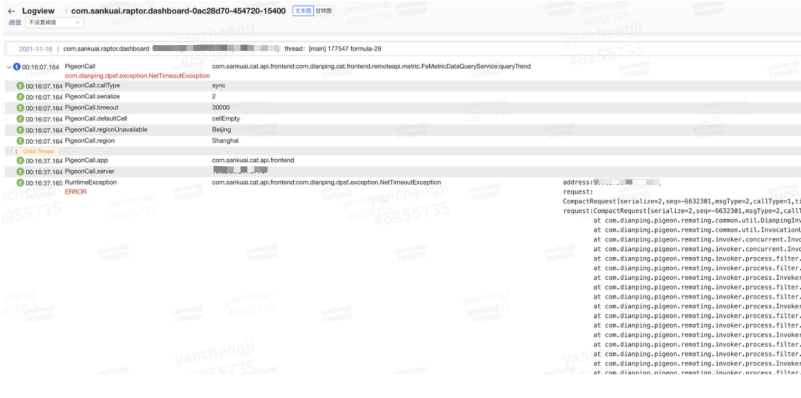
4. 总结与展望

总结—可观测案例分析

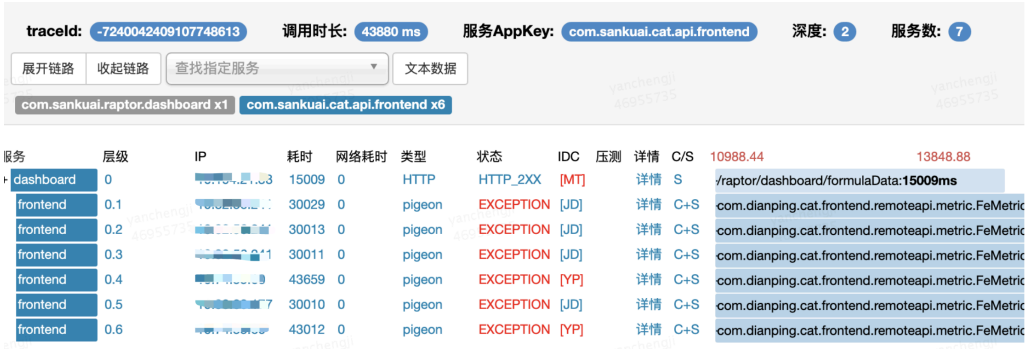
1. 收到告警，看到服务有报错



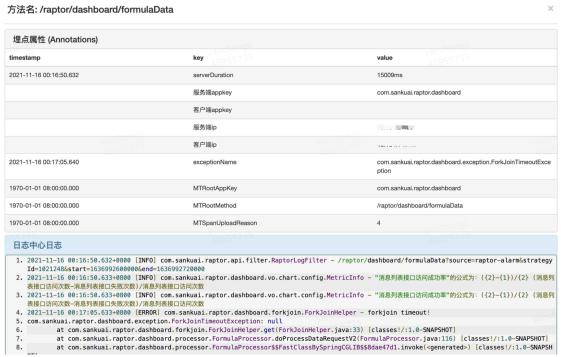
2. 打开链接，看到机器分布的指标数据，进行初步定位



3. 通过Examplar关联到内部链路，看到报错的源头是RPC调用



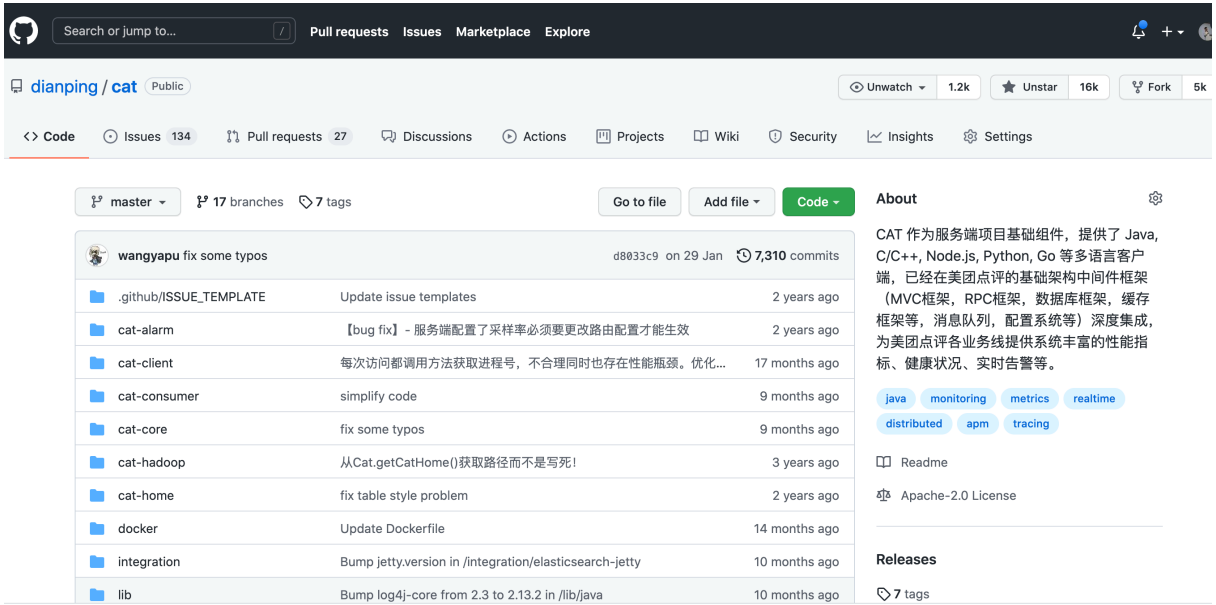
4. 同时使用关联到的Trace作为补充，定位源头服务



5. 通过Trace关联的日志，找到报错的根本原因

展望—未来规划与挑战

- Metrics、Tracing、Logging持续打通
- 兼容更多主流协议，例如：PromQL
- Paas、Iaas平台可观测性产品建设
- 可观测体系的云原生架构改造
- 开源建设，任重道远
  - https://github.com/dianping/cat



谢谢